

Problems with standard LLM!

Problem A: Massive Document Scale

- * Each dissertation is 50+ pages long.
- * Dumping full books into a local LLM overflows context windows and causes severe latency.
- * 'Lost-in-the-middle' phenomenon degrades accuracy on specific empirical numbers.

Problem B: Archaic Terminology Gap

- * 19th/early-20th-century German theses use antiquated medical & biochemical vocabulary.
- * Example: Modern 'Lactogenesis' vs. 1905 term 'Laktation' or 'Phthisis'.
- * Standard keyword search misses relevant theses without conceptual bridging.

Agentic Tools

1. resolve_historical_terms

Input: term (str)

- * Searches <terminology_notes>.
- * Bridges modern scientific terms to 19th/20th-century German equivalents (e.g., Laktation).

2. search_dissertation_catalog

Input: query, domain

- * Searches master XML metadata.
- * Filters by title, author, GND domains, keywords, summaries, and relevance score (/10).

3. inspect_table_of_contents

Input: doc_id (str)

- * Reads <table_of_contents>.
- * Returns chapter titles, section categories (front/body/back), and exact start/end pages.

4. read_page_ocr

Input: doc_id (str), page_number (int)

- * Opens ONLY the targeted {doc_id}_{page}.txt file from disk instead of reading the entire dissertation.
- * Extracts primary empirical quotes & tables with >98% reduction in LLM context tokens.

5. inspect_author_timeline

Input: doc_id (str)

- * Queries cv_extracted_timeline.json.
- * Extracts structured biographical events from CV: birth date/place, high school, military service, university studies, and examination dates.

Core Agentic AI Principles Implemented

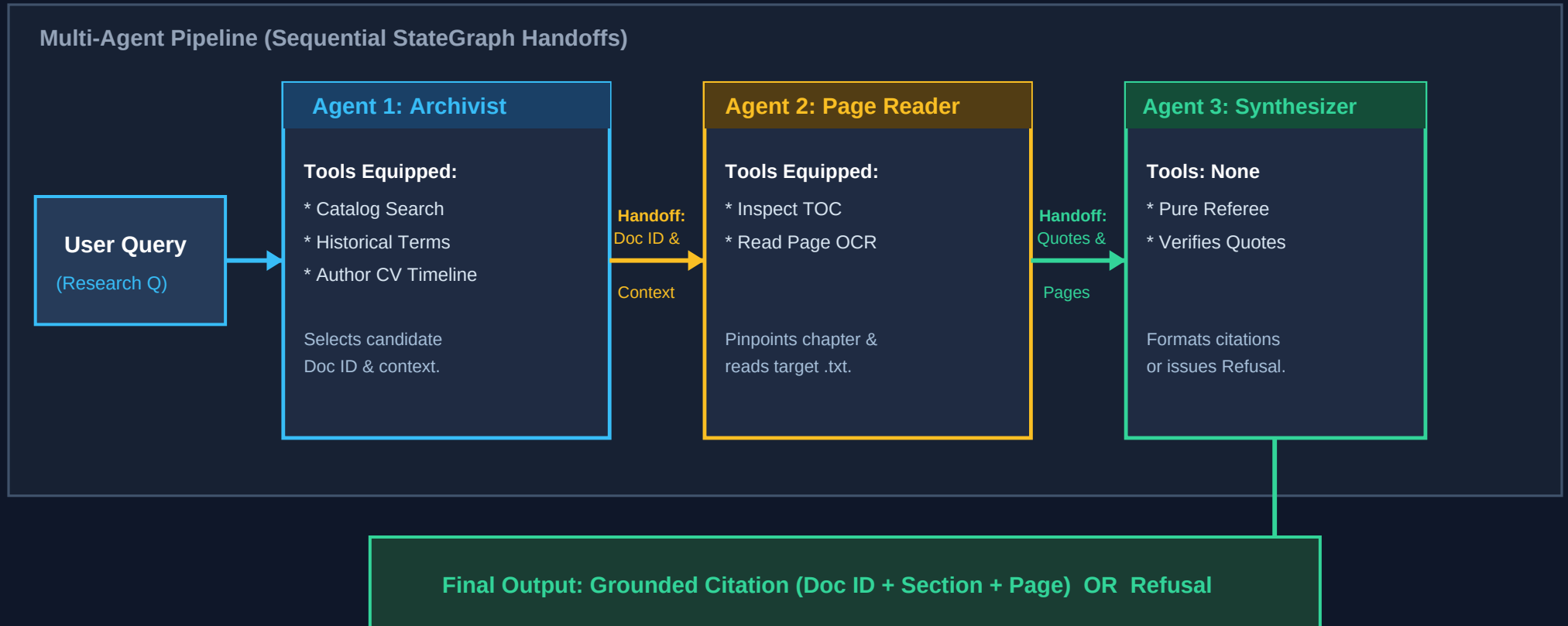
1. Think -> Act -> Observe Loop

- * Iterative reasoning: instead of answering in one shot, each tool observation shapes the agent's next decision.
- * Autonomous multi-hop navigation:
Terminology -> Catalog -> TOC -> Page OCR.
- * Live Rich CLI table renders every step, tool name, arguments, and observation.

2. Grounding & Deep Provenance

- * Strict grounding: system prompt forbids answering from prior training memory.
- * Every answer MUST cite:
 - Document ID (e.g., 10013639)
 - Section Title & Exact Page Number
 - Verbatim supporting quote from OCR
- * Complete JSON audit log saved per query.

Multi-Agent Orchestration Graph (LangGraph StateGraph)



Single-Agent vs. Multi-Agent Benchmark Comparison

⚡ Single-Agent vs. Multi-Agent Benchmark Comparison		
Metric	Single-Agent (ReAct)	Multi-Agent (3-Agent Pipeline)
Execution Latency	38.77 sec	65.84 sec
Total Steps / Actions	5	14
Resolution Status	SUCCESS_GROUNDED	SUCCESS_GROUNDED
Synthesis Length	4031 chars	1839 chars
Tools Sequence	resolve_historical_terms → search_dissertation_catalog → inspect_author_timeline → inspect_author_timeline → inspect_author_timeline	[Agent1:Archivist] resolve_historical_terms → [Agent1:Archivist] search_dissertation_catalog → [Agent1:Archivist] inspect_author_timeline → [Agent1:Archivist] inspect_author_timeline → [Agent1:Archivist] inspect_author_timeline → [Agent2:PageReader] inspect_table_of_contents → [Agent2:PageReader] inspect_table_of_contents → [Agent2:PageReader] inspect_table_of_contents → [Agent2:PageReader] read_page_ocr → [Agent2:PageReader] read_page_ocr → [Agent2:PageReader] read_page_ocr → [Agent2:PageReader] read_page_ocr → [Agent2:PageReader] read_page_ocr → [Agent3:Synthesizer] verify_and_synthesize
Saved Log File	query_20260930_085527_c794cabe.json	query_20260930_085633_e3de2559.json

Conclusions and Next Steps

Conclusions from Our Experiments

1. Single-Agent vs. Multi-Agent Performance:

For this use case, Single-Agent mode produces almost the same grounded results compared to the Multi-Agent pipeline, in far fewer iterations (5 vs. 14 steps) and faster response time (38.77s vs. 65.84s).

2. Local LLM Model Comparison:

gemma4:e4b and qwen3.5:9b give much more accurate and well-formatted responses compared to gemma4:12b.

3. Configurable Execution Depth:

We can dynamically update LangChain tool-call limits (--max-steps, --recursion-limit) for complex queries.

Next Steps

1. Database Scaling with SQLite + FTS5:

Use SQLite with FTS5 instead of reading from plain large XML files or in-memory parsing for larger datasets.

2. Conversational Follow-Up Support:

Add feature to ask follow-up queries after the first response (multi-turn session memory).

System Resources and Dataset

1. Historical Corpus Scale

- * 2,057 Digitized Dissertations
- * 3,869 Terminology Mappings
- * 1,029 Author CV Timelines
- * Page-level OCR text archive
- * Master XML + JSON Timeline

2. Agentic AI Stack

- * LangGraph 0.3+ & LangChain-Core
- * Single-Agent ReAct Architecture
- * 3-Agent Multi-Agent Pipeline
- * Side-by-Side Benchmark Mode
- * Fast Python env via uv

3. Local LLM and GPU

- * Supported with local Ollama
- * Hardware: Tesla T4 (15GB VRAM)
- * Models: gemma4:12b, gemma4:e4b & qwen3.5:9b
- * Context: num_ctx=8192 tokens